

ΥΠΟΛΟΓΙΣΤΙΚΗ ΝΟΗΜΟΣΥΝΗ

1^Η ΑΣΚΗΣΗ ΓΙΑ ΤΟ ΑΚΑΔΗΜΑΪΚΟ
ΕΤΟΣ 2024-2025

ΟΜΑΔΑ 5380-5226-5384-5228

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΚΩΝΣΤΑΝΤΙΝΟΣ ΖΩΗΣ, ΑΜ:5226

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΙΑΝΟΥΑΡΙΟΣ 2024

1 ΕΙΣΑΓΩΓΗ

Η παρούσα αναφορά συγγράφεται στα πλαίσια της πρώτης άσκησης του Project του μαθήματος της Υπολογιστικής Νοημοσύνης για το ακαδημαϊκό έτος 2024-2025 από τους φοιτητές Γιάννη Φίλλη (ΑΜ: 5380) και Κωνσταντίνο Ζώη (ΑΜ: 5226).

Σκοπός της συγκεκριμένης άσκησης ήταν να δημιουργήσουμε 2 μοντέλα MLP με 2 και 3 κρυμμένα επίπεδα και να μελετήσουμε πως μεταβάλλεται η γενικευτική ικανότητα, μεταβάλλοντας διάφορες παραμέτρους.

2 ΚΑΤΑΓΡΑΦΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ ΓΙΑ ΤΟ ΡΤ2

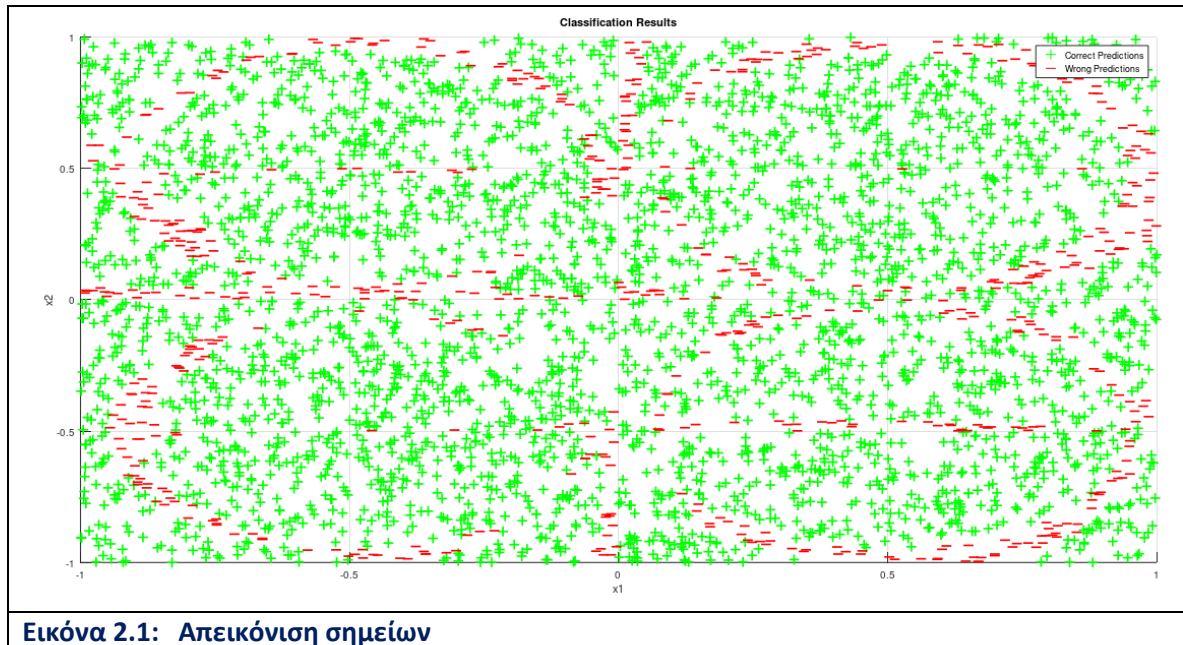
Στον παρακάτω πίνακα φαίνεται πως μεταβάλλεται η γενικευτική ικανότητα (ποσοστό επιτυχίας στο σύνολο ελέγχου) καθώς μεταβάλλουμε διάφορες παραμέτρους.

H1	H2	Ομάδες παραδειγμάτων B	Συνάρτηση ενεργοποίησης H2	Γενικευτική Ικανότητα (%)
2	4	20	Υπερβολική εφαπτομένη	35.625
4	6	20	Υπερβολική εφαπτομένη	37.0
6	6	20	Υπερβολική εφαπτομένη	57.6
12	6	20	Υπερβολική εφαπτομένη	44.575
8	4	20	Υπερβολική εφαπτομένη	47.575
20	40	20	Υπερβολική εφαπτομένη	84.225
2	4	20	relu	44.925003
4	6	20	relu	36.475
6	6	20	relu	54.525
12	6	20	relu	65.425
8	4	20	relu	43.95
20	40	20	relu	78.95
2	4	200	Υπερβολική εφαπτομένη	47.4
4	6	200	Υπερβολική εφαπτομένη	52.875
6	6	200	Υπερβολική εφαπτομένη	40.525
12	6	200	Υπερβολική εφαπτομένη	44.525
8	4	200	Υπερβολική εφαπτομένη	53.100002
20	40	200	Υπερβολική εφαπτομένη	29.15
2	4	200	relu	30.89998
4	6	200	relu	48.125
6	6	200	relu	30.89998
12	6	200	relu	30.89998
8	4	200	relu	30.89998
20	40	200	relu	22.6

Με βάση αυτές τις δοκιμές, μπορούμε να παρατηρήσουμε πως τα αποτελέσματα είναι παρόμοια είτε χρησιμοποιούμε την υπερβολική εφαπτομένη είτε την relu. Ωστόσο, στην δεύτερη περίπτωση όπου το B είναι 200, βλέπουμε πως η υπερβολική εφαπτομένη έχει πολύ καλύτερα αποτελέσματα από την relu.

Με την χρήση μεγαλύτερου αριθμού B παρατηρούμε πως στις δοκιμές με την υπερβολική εφαπτομένη βελτιώνεται η γενικευτική ικανότητα ενώ με την χρήση της relu, μειώνεται.

Παρακάτω ακολουθεί το γράφημα με τα παραδείγματα από το δίκτυο με την καλύτερη γενικευτική ικανότητα (84.225%)



Εικόνα 2.1: Απεικόνιση σημείων

3 ΚΩΔΙΚΑΣ

Στο συγκεκριμένο σημείο θα επεξηγήσουμε λίγο τον κώδικα ώστε να μην υπάρχουν ασάφειες. Αρχικά, για να τρέξουμε τον κώδικα γράφουμε στο τερματικό **javac PT2.java** για να γίνει compile και έπειτα **java PT2**. Το ίδιο ισχύει και για το PT3. Όπως ζητάει και η εκφώνηση οι παράμετροι ορίζονται στην αρχή του προγράμματος. Οπότε όλες οι δοκιμές για τις διάφορες τιμές των παραμέτρων (αριθμό νευρώνων σε κάθε επίπεδο, συνάρτηση ενεργοποίησης, αριθμός batches) έγιναν από εκεί. Αν και ζητάγε με την εντολή define, καθώς αυτή δεν υποστηρίζεται στην java, αντίστοιχα εμείς τα έχουμε ορίσει ως public static final.

Όσον αφορά την κωδικοποίηση των δεδομένων, χρησιμοποιήσαμε την κωδικοποίηση 1-out-of- p που συζητήθηκε και κατά την διάρκεια των διαλέξεων.

```
Scanner scanner = new Scanner(input) {  
  
    while (scanner.hasNextLine()) {  
        String line = scanner.nextLine().trim();  
        String[] parts = line.split("\\s+");  
        if (parts.length == 3) {  
            float x1 = Float.parseFloat(parts[0].replace(",", "."));  
            float x2 = Float.parseFloat(parts[1].replace(",", "."));  
            String label = parts[2];  
  
            pointData.add(new float[]{x1, x2});  
            labelData.add(label);  
        }  
    }  
} catch (IOException e) {  
    System.out.println("An error occurred\n" + e.getMessage());  
}  
  
for (int i = 0; i < pointData.size(); i++) {  
    testX[i] = pointData.get(i);  
}  
for (int i = 0; i < labelData.size(); i++) {  
    if (labelData.get(i).equals("C1")) {  
        testT[i] = new float[]{1,0,0,0};  
    }  
    else if (labelData.get(i).equals("C2")) {  
        testT[i] = new float[]{0,1,0,0};  
    }  
    else if (labelData.get(i).equals("C3")) {  
        testT[i] = new float[]{0,0,1,0};  
    }  
    else {  
        testT[i] = new float[]{0,0,0,1};  
    }  
}
```

Εικόνα 3.1: Κωδικοποίηση δεδομένων

Έπειτα, όταν θέλουμε να ελέγξουμε την ικανότητα γενίκευσης βρίσκουμε ποιος νευρώνας έχει την μέγιστη τιμή και αν πχ την μέγιστη τιμή την έχει ο νευρώνας 1 τότε είναι κατηγορίας C1.

```
public void testGeneralAbility(){
    String fileName = "PT2-" + H1 + "-" + H2 + "-" + B + "-" + layers[2].getActivationFunction() + ".txt";
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
        int correctPredictions = 0;
        int testMade = 0;
        String trueClass = " ";
        String predictedClass = " ";
        for(int i=0; i<testX.length; i++){
            float[] outputOfMPLP = forward_pass(testX[i], d, testT[i], K);
            int labelOfOutput = findMaxIndex(outputOfMPLP);
            int trueLabel = findMaxIndex(testT[i]);
            testMade++;

            if(labelOfOutput == trueLabel){
                correctPredictions++;
            }
            trueClass = convertToStringClass(trueLabel);
            predictedClass = convertToStringClass(labelOfOutput);
            float x1 = testX[i][0];
            float x2 = testX[i][1];
            writer.write(x1 + " " + x2 + " " + predictedClass + " " + trueClass + "\n");
        }
        float accuracy = (correctPredictions/(float) testSetSize)*100;
        System.out.println("The generalization ability is " + accuracy + " % ");
    }
    catch (IOException e) {
        System.err.println("Error while writing to file: " + e.getMessage());
    }
}
```

Εικόνα 3.2: Έλεγχος γενίκευσης

Παρακάτω έχουμε την μέθοδο που υλοποιεί των αλγόριθμο του gradient descent.

```
public void gradientDescent(){
    int batchIteration = 1;
    int epochs = 0;
    while (conditionEvaluation(epochs)){
        squaredErrorOfEpoch = 0; // initializing again the squared error

        for(int i=0; i<trainingSetSize; i++){
            backprop(x[i], d, t[i], K);
            updateGradients(layers[1],1); // first hidden layer
            updateGradients(layers[2],2); // second hidden layer
            updateGradients(layers[3],3); // output layer
            if (batchIteration == B){ // a batch has been completed
                updateWeightsOfLayer(layers[1]); // first hidden layer
                updateWeightsOfLayer(layers[2]); // second hidden layer
                updateWeightsOfLayer(layers[3]); // output layer
                resetGradientArrays();

                deleteDeltasOfLayers();
                batchIteration = 0;
            }
            batchIteration++;
        }
        float totalError = calculateError();
        currentError[epochs] = squaredErrorOfEpoch/2;
        System.out.println("Epoch: "+epochs+" Total error: "+totalError);
        epochs++;
    }
}
```

Εικόνα 3.3: Κώδικας gradient descent

Με το βρόγχο while, ελέγχουμε τις συνθήκες που απαιτούνται και δίνονται από την εκφώνηση. Στην αρχή λοιπόν κάθε εποχής, μηδενίζουμε το error. Έπειτα με τον βρόγχο for περνάμε από όλα τα δεδομένα εισόδου. Εκτελούμε το backpropagation(όπου μέσα σε αυτό γίνεται και το forward pass) και ανανεώνουμε τα gradients των νευρώνων. Με τον έλεγχο if, ελέγχουμε εάν συμπληρώσαμε ένα batch και αν έχει συμβεί αυτό, τότε ανανεώνουμε τα βάρη των νευρώνων των κρυμμένων επιπέδων και του επιπέδου εξόδου και αρχικοποιούμε ξανά τα gradients των νευρώνων και τους προσωρινούς πίνακες.

Στο τέλος υπολογίζουμε το error της εποχής και το κάνουμε print.

Το πρόγραμμα DataGeneration.java χρησιμοποιήθηκε για την αποθήκευση του συνόλου ελέγχου και εκπαίδευσης στα αρχεία test_set.txt και training_set.txt.

4 ΚΑΤΑΓΡΑΦΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ ΓΙΑ ΤΟ ΡΤ3

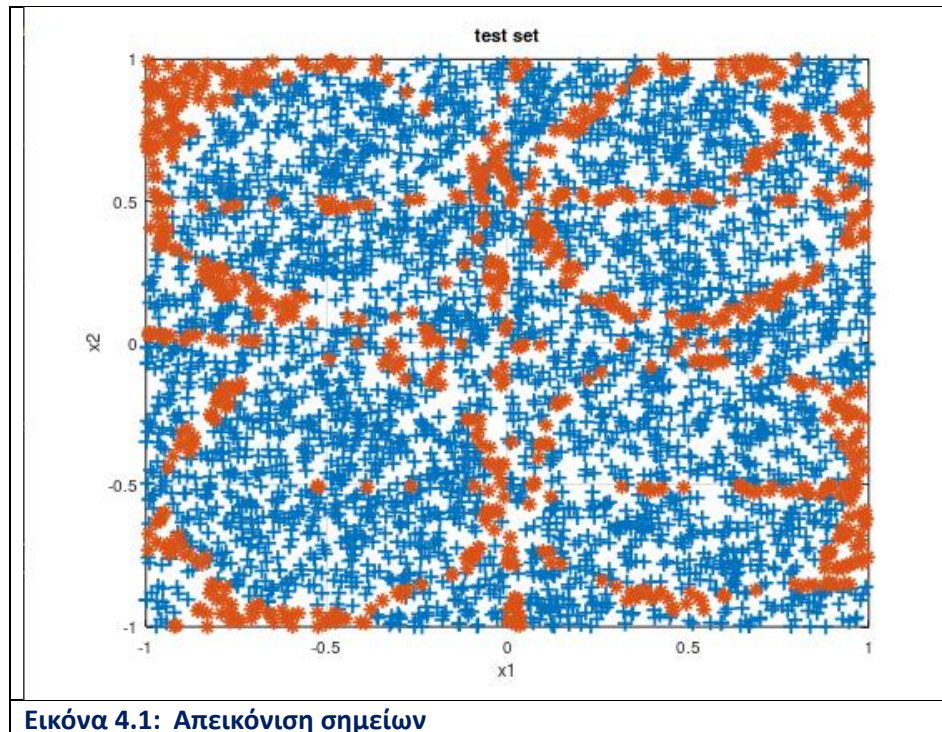
Στον παρακάτω πίνακα φαίνεται πως μεταβάλλεται η γενικευτική ικανότητα (ποσοστό επιτυχίας στο σύνολο ελέγχου) καθώς μεταβάλλουμε διάφορες παραμέτρους.

H1	H2	H3	Ομάδες παραδειγμάτων B	Συνάρτηση ενεργοποίησης H3	Γενικευτική Ικανότητα (%)
2	3	4	20	Υπερβολική εφαπτομένη	43.175
2	4	4	20	Υπερβολική εφαπτομένη	38.0
4	6	8	20	Υπερβολική εφαπτομένη	34.25
8	6	4	20	Υπερβολική εφαπτομένη	44.825
4	10	8	20	Υπερβολική εφαπτομένη	38.45
10	10	8	20	Υπερβολική εφαπτομένη	46.85
20	24	16	20	Υπερβολική εφαπτομένη	56.25
16	24	32	20	Υπερβολική εφαπτομένη	65.275
20	40	8	20	Υπερβολική εφαπτομένη	51.475
2	3	4	20	relu	42.649998
2	4	4	20	relu	47.350002
4	6	8	20	relu	47.575
8	6	4	20	relu	41.025
4	10	8	20	relu	36.575
10	10	8	20	relu	35.525
20	24	16	20	relu	70.65
16	24	32	20	relu	55.225002
20	40	8	20	relu	80.025
2	3	4	200	Υπερβολική εφαπτομένη	47.075
2	4	4	200	Υπερβολική εφαπτομένη	33.774998
4	6	8	200	Υπερβολική εφαπτομένη	26.925001
8	6	4	200	Υπερβολική εφαπτομένη	37.7
4	10	8	200	Υπερβολική εφαπτομένη	34.625
10	10	8	200	Υπερβολική εφαπτομένη	48.275
20	24	16	200	Υπερβολική εφαπτομένη	18.025
16	24	32	200	Υπερβολική εφαπτομένη	19.225
20	40	8	200	Υπερβολική εφαπτομένη	36.85
2	3	4	200	relu	30.899998
2	4	4	200	relu	30.899998
4	6	8	200	relu	30.899998
8	6	4	200	relu	30.899998
4	10	8	200	relu	19.225
10	10	8	200	relu	30.899998
20	24	16	200	relu	30.899998
16	24	32	200	relu	21.1
20	40	8	200	relu	30.899998

Παρατηρούμε πως αλλάζοντας την συνάρτηση ενεργοποίησης δεν έχουμε και πολύ μεγάλη μεταβολή στην γενικευτική ικανότητα του δικτύου.

Ωστόσο, αυξάνοντας το πλήθος των ομάδων παρατηρούμε πως η γενικευτική ικανότητα του δικτύου γίνεται αρκετά χαμηλότερη σε ορισμένες περιπτώσεις.

Από τα αποτελέσματα που συγκεντρώσαμε από τα πειράματα, παρατηρούμε ότι το δίκτυο με τη μεγαλύτερη γενικευτική ικανότητα (80.025 %) είναι αυτό όπου $H1 = 20$, $H2 = 40$, $H3 = 8$ με συνάρτηση ενεργοποίησης relu στο τρίτο κρυμμένο επίπεδο και αριθμό batches $N/200$. Παρακάτω βλέπουμε το γράφημα των παραδειγμάτων του συνόλου ελέγχου. Με το σύμβολο «+» έχουμε αποτυπώσει τα σημεία που ταξινομούνται στη σωστή κατηγορία ενώ με το «*» τα σημεία που δεν ταξινομούνται σωστά.



Εικόνα 4.1: Απεικόνιση σημείων

Σε σύγκριση των δύο δικτύων, δεν φαίνεται το ΠΤ3 να υπερτερεί του ΠΤ2. Τα ποσοστά γενίκευσης είναι παρόμοια και μπορεί να φτάσει στα ίδια και το ΠΤ2. Θα μπορούσαμε να πούμε ότι το ΠΤ2 είναι επαρκές για αυτό το πρόβλημα και λίγο καλύτερο από το ΠΤ3 καθώς έχει ελαφρώς καλύτερα αποτελέσματα για λιγότερους αριθμούς νευρώνων στα επίπεδα.